Modélisation et Commande de Systèmes

TD 4 : Identification de systèmes

Objectifs :
Savoir :
— Identifier un modèle d'état à partir de données
— Se familiariser avec la Toolbox Ident de Matlab

Vous aurez besoin d'installer la `Toolbox System Identification`

1 Données artificielles

1.1 Le script `ident_data_generation.m` fourni permet de générer des données entrée-sortie artificielles à partir d'un exemple numérique. Lancer ce script et tracer les signaux d'entrée et de sortie.

1.2 Ajouter les lignes suivantes au script pour identifier un modèle du système. Quel est l'intérêt de la figure qui s'affiche ?

```
%% 1. Créer un objet iddata
data = iddata(y, u, Ts);

%% 2. Appliquer l'algo n4sid pour identifier un modèle d'état
nx_max  = 10; % choix ordre
model = n4sid(data, 1:nx_max);
```

1.3 Comparer le modèle identifié au modèle générateur des données. Les deux modèles sont-ils égaux ? proches ? Sinon, pourquoi ?

Note : Les paramètres du modèle estimé peuvent être individuellement obtenus via les commandes `model.a`, `model.b`, `model.c`, `model.d`.

1.4 Proposer deux méthodes pour comparer les paramètres du modèle identifié à ceux du vrai modèle.

1.5 Dans une situation réelle on ne dispose pas d'un « vrai modèle » auquel on pourrait comparer directement le modèle identifié. Pour évaluer la performance du modèle, on mesure alors sa capacité de prédiction sur un autre jeu de données (que celles ayant servi à son estimation). C'est la méthode de la *validation croisée*.

Le code suivant génère de nouvelles données entrée-sortie (à partir du même système) en vue de cette validation.

```
%% 3. Validation sur de nouvelles données
u_val = randn(N,1); % entrée comme une réalisation de bruit blanc
y_val = lsim(sys_true, u_val); % génération de la sortie correspondante
```

```
y_val = y_val + sigE*randn(size(y_val)); % Ajout de bruit de mesure

data_val = iddata(y_val, u_val, Ts); % création d'un objet iddata
```

On peut alors représenter la sortie réelle et la sortie simulée à partir du modèle identifié. En plus de la comparaison graphique, on peut s'aider d'un critère quantificatif pour mesurer la correspondance entre les deux signaux (critère FIT calculé dans le code).

```
[data_model, FIT] = compare(data_val, model); %
```

```
%% Tracer des figures
figure;
t = data_val.SamplingInstants;
plot(t, data_val.OutputData, 'b', 'LineWidth', 2); hold on;
plot(t, data_model.OutputData, 'r--', 'LineWidth', 2);
legend('Measured', 'Model');
set(gca,'fontsize',15)
xlabel('time','FontSize',17)
ylabel('outputs','FontSize',17)
title(['Comparison␣$y$␣vs␣$\hat{y}$:␣FIT␣=␣' num2str(FIT)],...
    'interpreter', 'latex', 'fontweight','bold','fontsize',14)
grid on;
```

Etudier la performance du modèle en fonction de différents niveaux de bruits.

Pour cela, tracer l'évolution du FIT et d'une certaine erreur paramétrique en fonction du rapport signal sur bruit (SNR).

2 Données expérimentales

On considère maintenant la modélisation d'un système réel à partir de données expérimentales issues d'une vraie expérience. Le système considéré est un bras de robot flexible [1] dont une possible description fonctionnelle est représentée sur la figure 1. L'entrée u est le couple de rappel développé par le moteur à courant continu supportant le bras, la sortie y est l'accélération du bras flexible.

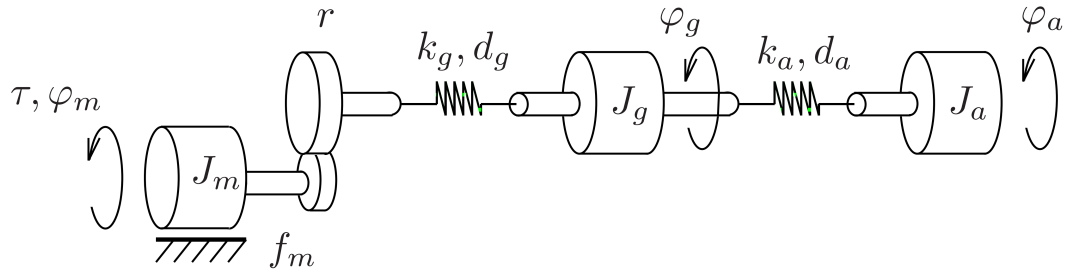

FIGURE 1 – Schéma de principe d'un joint de bras robotique (un exemple).

1. La figure ne présente qu'un exemple illustratif de bras de robot. Ce n'est pas forcément celui dont les données sont issues.

2.1 A partir des données contenues dans le fichier `data_robot_arm.mat` construire un modèle de ce système et procéder à sa validation. *Pour réaliser une validation croisée, on pourra, par exemple, partionner les données en deux groupes : un pour faire l'estimation du modèle et l'autre pour sa validation.*

2.2 On considère enfin un autre jeu de données issu de notre robot bleu dans lequel l'entrée u est la tension du moteur et la sortie y est la position linéaire. Une particularité de ces données est qu'elles ont été obtenues en boucle fermée avec un correcteur proportionnel $u = -k(r - y)$, où r est un signal de consigne sinusoïdal et $k = 1$. A votre avis, pourquoi était-il nécessaire de passer en boucle fermée pour générer ces données ?

2.3 On cherche un modèle temps-continu du système en boucle ouverte.

(a) Quelle est l'entrée du système en BF et quelle en est la sortie ?

(b) Exprimer les matrices d'un modèle d'état $(A_{\text{bf}}, B_{\text{bf}}, C_{\text{bf}}, D_{\text{bf}})$ de la BF en fonction de celles (A, B, C, D) de la BO.

(c) Estimer un modèle de la BF à partir des données contenues dans le fichier `data_robot_MakeBlock.mat`.

(d) En déduire un modèle de la BO. L'un des pôles du système en BO doit être proche de 1. Pourquoi ?

(e) Recalculer le modèle de BO de manière à forcer ce pôle à 1.
Cela peut se faire en multipliant la matrice B_{bf} par un coefficient avant la transformation.

(f) En utilisant la commande `d2c` convertir le modèle obtenu en un modèle à temps continu.

3 Annexes

Il existe beaucoup d'autres méthodes d'identification implémentées dans la Toolbox System Identification, notamment les méthodes entrée-sortie (non étudiées dans le cadre de ce cours).

3.1 Rappels sur la procédure d'identification

Comme vu en cours, l'identification de système est une procédure itérative qui se fait en plusieurs étapes :

(i) **Choix d'une structure de modèle :**

$$M(\theta): \quad \hat{y}(t|\theta) = \varphi(t,\theta)^\top \theta, \quad \theta \in \mathbb{R}^d \tag{1}$$

où $\hat{y}(t|\theta)$ est la prédiction de la sortie du système donnée par le modèle pour une valeur θ donnée des paramètres, et $\varphi(t,\theta)$ est le régresseur.
Choix d'un type de modèle (ce qui induit une forme spécifique pour $\varphi(t,\theta)$) et détermination de ses paramètres structuraux (ordres des polynômes intervenant dans ses fonctions de transfert).

(ii) **Estimation d'un vecteur de paramètres :**
Déterminer le vecteur de paramètres qui minimise une mesure de l'erreur de prédiction,

$$\hat{\theta}_N = \arg\min_{\theta \in \mathbb{R}^d} \frac{1}{N} \sum_{t=1}^{N} \ell(\varepsilon(t,\theta)) \tag{2}$$

avec $\varepsilon(t,\theta) = y(t) - \hat{y}(t|\theta)$.
Commandes Matlab disponibles selon la structure de modèle choisie : `arx`, `armax`, `oe`, `bj` et beaucoup d'autres en fonction du mode d'estimation (récursif ou non).

(iii) **Validation du modèle estimé :**
Une fois un modèle estimé, il convient de vérifier sa fiabilité à travers différents tests. Une méthode de test courante est celle dite de la *validation croisée*. Elle consiste à évaluer la capacité du modèle estimé à récréer les données de validation (idéalement indépendantes de celles qui ont été utilisées pour l'estimation) par prédiction. Ci-dessous trois tests de validation sont décrits :

— *Mesure de l'erreur de prédiction* : Il existe plusieurs façons de quantifier empiriquement la performance d'un modèle. Un exemple de mesure de performance est la fonction FIT définie par

$$\mathrm{FIT} = \left[1 - \frac{\|\hat{y} - y\|_2}{\|y - \bar{y}\|_2}\right] \times 100$$

où y est le vecteur des sorties mesurées, $\hat{y}$ est le vecteur des sorties produites par le modèle, $\bar{y}$ est un vecteur contenant la moyenne des sorties mesurées. Le critère FIT est exprimé en pourcentage. La qualité du modèle est d'autant meilleure que ce nombre est proche de 100. Cependant, la valeur de ce critère étant dépendante de la quantité de bruit affectant les données, il ne permet pas de faire une validation définitive mais peut servir à comparer plusieurs modèles sur un même jeu de données.
Commande Matlab : `compare`.

— *Test de blancheur :*
Ce test est fondé sur le principe que plus $\hat{\theta}_N$ est proche de θ°, plus la séquence de résidus $\{\varepsilon(t,\hat{\theta}_N)\}$, où $\varepsilon(t,\hat{\theta}_N) = y(t) - \hat{y}(t|\hat{\theta}_N)$, devrait tendre vers un bruit blanc (cf. Cours, S.61 ou S.86 sous les hypothèses données S.56).
Commande Matlab : `resid`.

— *Test d'indépendance :*
Pour un modèle performant, le résidu $\varepsilon(t) = \varepsilon(t,\hat{\theta}_N)$ doit (dans des conditions de boucle ouverte) être statistiquement indépendant des entrées passées

$$\Rightarrow \quad \hat{R}^N_{\varepsilon u}(\tau) = \frac{1}{N}\sum_{t=1}^{N} \varepsilon(t)u(t-\tau) \approx 0 \quad \text{pour tout } \tau \geq 0.$$

Commande Matlab : `resid`.

3.2 Quelques commandes de la System Identification Toolbox

— Pour visualiser toutes les fonctions de la Toolbox, taper la commande
`help ident`

— Création, manipulation de modèles/données :
`idpoly`, `idinput`, `iddata`, `sim`, ...

— Méthodes d'estimation non paramétrique :
`impulseest`, `cra`, `spa`, `etfe`, `tfest`, `ssest`...

— Sélection de structure de modèle :
`delayest`, `arxstruc`, `selstruc`, `struc`,
`n4sid`, `pem`, `fpe`, `aic`, ...

— Méthodes d'estimation paramétrique :
`ar`, `arx`, `armax`, `oe`, `bj`, `pem`, `polyest`, ...
`iv`, `iv4`, ...
`recursiveARX` , `recursiveARMAX`, `recursiveOE`, `recursiveBJ`, `rpem`, ...
`n4sid`, ...

— Outils de lecture des propriétés du modèle :
`present`, `getpvec`, `getcov`, `setpvec`, ...

— Méthodes de validation :
`resid`, `compare`, `predict`, ...

— Lancement de la fenêtre graphique :
`ident` ou `systemIdentification` en fonction de la version Matlab